

Felipe Fazio da Costa, RA: 23.00055-4  
João Gabriel Fioruci Roberto, RA: 23.00617-0  
Gabriel Rodrigues Marques, RA: 23.00578-5  
Fábio Sadao Sato, RA: 22.00984-0

## **Projeto Semestral: Carro de Bombeiro Teleoprado**

Brasil

2026

Felipe Fazio da Costa, RA: 23.00055-4  
João Gabriel Fioruci Roberto, RA: 23.00617-0  
Gabriel Rodrigues Marques, RA: 23.00578-5  
Fábio Sadao Sato, RA: 22.00984-0

## **Projeto Semestral: Carro de Bombeiro Teleoprado**

Relatório técnico apresentado como requisito  
parcial para aprovação na disciplina EEN251  
— Projeto Integrador de Sistemas Embarcados  
do curso de Engenharia.

Centro Universitário Instituto Mauá de Tecnologia  
Engenharia de Computação / Eletrônica  
Disciplina: EEN251 — Projeto de Sistemas Embarcados

Brasil  
2026

# Resumo

Este relatório descreve o desenvolvimento de um protótipo de carro de bombeiro teleoprado utilizando microcontroladores Raspberry Pi Pico e comunicação via rádio frequência (NRF24L01). O sistema integra tração 4WD, sistema de combate a incêndio por bomba d'água e monitoramento de nível de reservatório. Foram aplicados conceitos de sistemas embarcados, comunicação sem fio e controle de periféricos via PWM. Os resultados demonstram a eficácia da plataforma RP2040 em aplicações de controle em tempo real e teleoperação robusta.

**Palavras-chave:** Raspberry Pi Pico. NRF24L01. MicroPython. Carro de Bombeiro. Sistemas Embarcados.

# Abstract

This report describes the development of a teleoperated fire truck prototype using Raspberry Pi Pico microcontrollers and radio frequency communication (NRF24L01). The system integrates 4WD traction, a water pump fire suppression system, and reservoir level monitoring. Concepts of embedded systems, wireless communication, and peripheral control via PWM were applied. The results demonstrate the effectiveness of the RP2040 platform in real-time control applications and robust teleoperation.

**Keywords:** Raspberry Pi Pico. NRF24L01. MicroPython. Fire Truck. Embedded Systems.

# Sumário

|            |                                                         |           |
|------------|---------------------------------------------------------|-----------|
| <b>1</b>   | <b>INTRODUÇÃO</b>                                       | <b>6</b>  |
| <b>1.1</b> | <b>Objetivos</b>                                        | <b>6</b>  |
| <b>2</b>   | <b>DESENVOLVIMENTO DO PROJETO</b>                       | <b>7</b>  |
| <b>2.1</b> | <b>Arquitetura e Diagramas de Blocos</b>                | <b>7</b>  |
| 2.1.1      | Subsistema Transmissor (Controle)                       | 7         |
| 2.1.2      | Subsistema Receptor (Veículo)                           | 7         |
| <b>2.2</b> | <b>Arquitetura de Hardware</b>                          | <b>8</b>  |
| 2.2.1      | Transmissor (Controle)                                  | 8         |
| 2.2.2      | Receptor (Veículo)                                      | 8         |
| <b>2.3</b> | <b>Comunicação RF (NRF24L01)</b>                        | <b>8</b>  |
| <b>2.4</b> | <b>Software e Firmware</b>                              | <b>9</b>  |
| 2.4.1      | Lógica de Movimentação (Tank Drive)                     | 9         |
| 2.4.2      | Segurança e Failsafe                                    | 9         |
| <b>2.5</b> | <b>Modelagem 3D e Estrutura Física</b>                  | <b>9</b>  |
| <b>2.6</b> | <b>Esquemas Elétricos e PCB</b>                         | <b>9</b>  |
| <b>3</b>   | <b>RESULTADOS E DISCUSSÃO</b>                           | <b>12</b> |
| <b>3.1</b> | <b>Protótipo do Veículo</b>                             | <b>12</b> |
| <b>3.2</b> | <b>Unidade de Controle Remoto</b>                       | <b>13</b> |
| <b>3.3</b> | <b>Integração Eletrônica e Sistema de Tração</b>        | <b>13</b> |
| <b>3.4</b> | <b>Análise de Desempenho</b>                            | <b>14</b> |
| <b>4</b>   | <b>CONCLUSÃO</b>                                        | <b>18</b> |
|            | <b>REFERÊNCIAS BIBLIOGRÁFICAS</b>                       | <b>19</b> |
|            | <b>APÊNDICES</b>                                        | <b>20</b> |
|            | <b>APÊNDICE A – CÓDIGO FONTE DO RECEPTOR</b>            | <b>21</b> |
|            | <b>APÊNDICE B – CÓDIGO FONTE DO TRANSMISSOR</b>         | <b>27</b> |
|            | <b>APÊNDICE C – MODIFICAÇÃO TÉCNICA DA BOMBA D'ÁGUA</b> | <b>35</b> |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>ANEXOS</b>                                           | <b>36</b> |
| <b>ANEXO A – ESQUEMA ELÉTRICO DO CARRO . . . . .</b>    | <b>37</b> |
| <b>ANEXO B – LAYOUT DA PCB DO CARRO . . . . .</b>       | <b>39</b> |
| <b>ANEXO C – ESQUEMA ELÉTRICO DO CONTROLE . . . . .</b> | <b>43</b> |
| <b>ANEXO D – LAYOUT DA PCB DO CONTROLE . . . . .</b>    | <b>45</b> |

# 1 Introdução

O projeto semestral da disciplina EEN251 — Projeto Integrador de Sistemas Embarcados propõe o desenvolvimento de um sistema embarcado completo, integrando hardware e software. O tema escolhido foi um carro de bombeiro teleoperado, que combina mobilidade, atuação remota e sensoriamento.

O uso de veículos teleoperados é comum em situações de risco, como o combate a incêndios, onde a segurança humana é prioridade. O protótipo desenvolvido busca simular essas funcionalidades em escala reduzida.

## 1.1 Objetivos

O objetivo geral deste trabalho é projetar e construir um veículo robótico controlado remotamente via radiofrequência, capaz de realizar manobras de locomoção e acionamento de uma bomba d'água, com monitoramento de status em tempo real.

Como objetivos específicos, destacam-se:

- Implementação de comunicação bidirecional robusta entre controle e veículo;
- Controle de tração diferencial (tank drive) para quatro motores;
- Sistema de alerta de nível de água com indicadores visuais e sonoros;
- Desenvolvimento de firmware em MicroPython para a plataforma Raspberry Pi Pico.

## 2 Desenvolvimento do Projeto

O desenvolvimento foi dividido em três frentes principais: hardware, software e integração de sistemas.

### 2.1 Arquitetura e Diagramas de Blocos

A Figura 1 apresenta a visão sistêmica do projeto, destacando a separação entre as unidades de controle e de atuação no veículo.

Figura 1 – Diagrama de Blocos do Sistema Completo

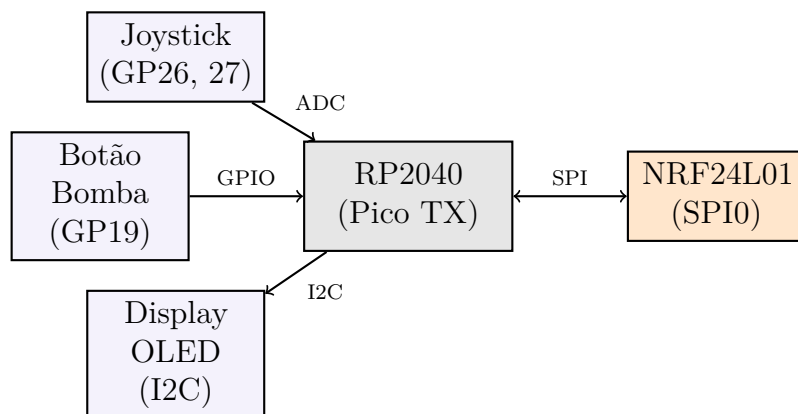


Fonte: Os autores (2026)

#### 2.1.1 Subsistema Transmissor (Controle)

O controle remoto (Figura 2) processa as entradas do joystick e botões através do Raspberry Pi Pico, transmitindo-as via NRF24L01.

Figura 2 – Fluxo de Dados do Transmissor



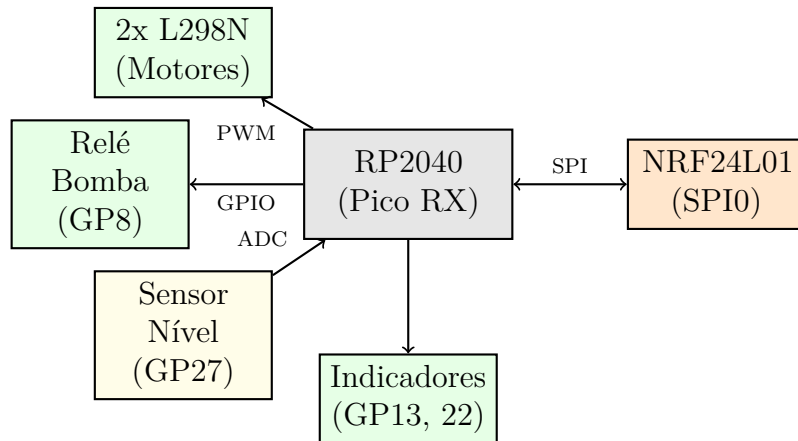
Fonte: Os autores (2026)

#### 2.1.2 Subsistema Receptor (Veículo)

O receptor (Figura 3) traduz os sinais RF em comandos de potência para os motores e gerencia os sensores de campo.



Figura 3 – Fluxo de Atuação do Receptor



Fonte: Os autores (2026)

## 2.2 Arquitetura de Hardware

O sistema é composto por dois nós principais: o Transmissor (Controle) e o Receptor (Veículo). Ambos utilizam o microcontrolador RP2040 ([RASPBerry PI LTD, 2026b](#)), presente na placa Raspberry Pi Pico ([RASPBerry PI LTD, 2026a](#)).

### 2.2.1 Transmissor (Controle)

O controle remoto utiliza um joystick analógico para comandos de direção e botões para acionamento da bomba. Um display OLED SSD1306 foi integrado para fornecer feedback visual ao operador.

### 2.2.2 Receptor (Veículo)

O veículo possui um chassi 4WD acionado por motores DC. O controle de potência é realizado por drivers L298N ([STMICROELECTRONICS, 2026](#)). O sistema possui uma mini bomba submersa acionada por relé.

## 2.3 Comunicação RF (NRF24L01)

A comunicação utiliza o módulo NRF24L01 ([NORDIC SEMICONDUCTOR, 2026](#)) operando em 2.4 GHz. Foi adotado o protocolo Enhanced ShockBurst. Todo o desenvolvimento do rádio foi baseado na biblioteca ([MICROPYTHON-NRF24L01..., 2026](#)).

## 2.4 Software e Firmware

Todo o código foi desenvolvido em MicroPython ([MICROPYTHON PROJECT, 2026](#)). O firmware gerencia a recepção de pacotes e o controle de periféricos.

### 2.4.1 Lógica de Movimentação (Tank Drive)

```
1 def drive(x, y):  
2     motor_esq = max(-100, min(100, y + x))  
3     motor_dir = max(-100, min(100, y - x))  
4     set_motor(motor_esq, motor_dir)
```

Listing 2.1 – Algoritmo de Tank Drive em MicroPython

### 2.4.2 Segurança e Failsafe

```
1 if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > 500:  
2     parar()           # Desliga motores por seguranca
```

Listing 2.2 – Logica de Watchdog

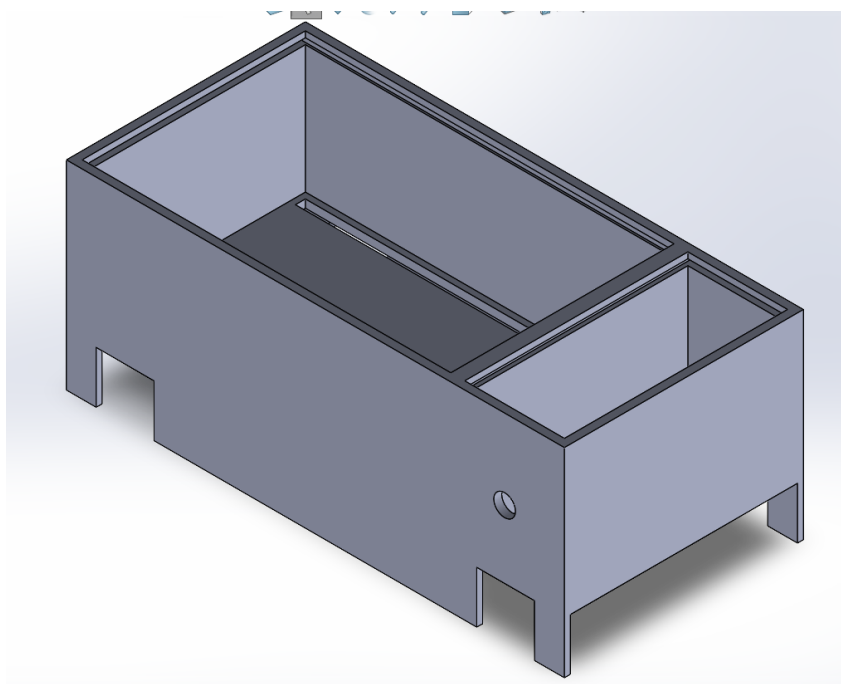
## 2.5 Modelagem 3D e Estrutura Física

A estrutura física foi projetada em CAD. A Figura 4 apresenta a renderização do veículo.

## 2.6 Esquemas Elétricos e PCB

A Figura 7 ilustra a visualização 3D da PCB do carro, enquanto a Figura 8 apresenta a visualização 3D da PCB do controle.

Figura 4 – Modelagem 3D do Carro de Bombeiro



Fonte: Os autores (2026)

Figura 5 – Base do Controle

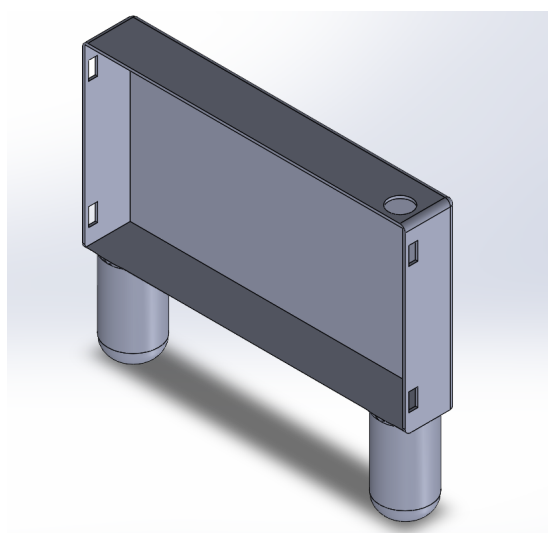
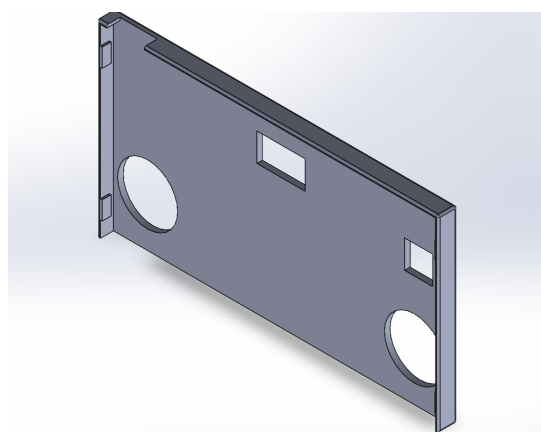
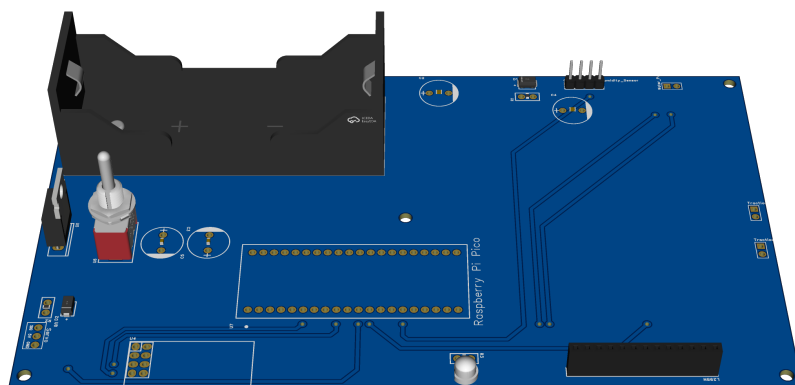


Figura 6 – Tampa do Controle



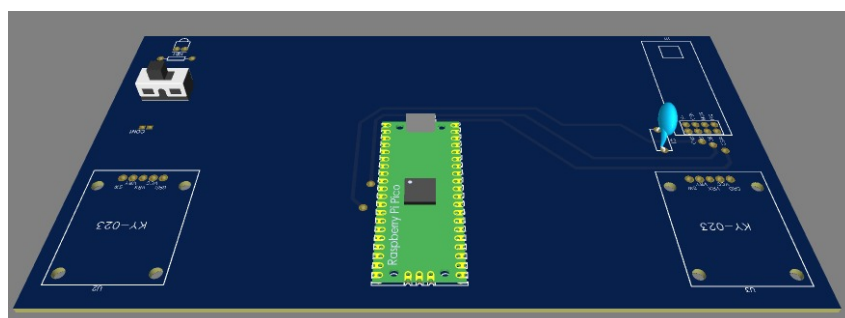
Fonte: Os autores (2026)

Figura 7 – Visualização 3D da PCB do Carro



Fonte: Os autores (2026)

Figura 8 – Visualização 3D da PCB do Controle



Fonte: Os autores (2026)

## 3 Resultados e Discussão

Neste capítulo são detalhados os resultados práticos da construção do protótipo, analisando a estrutura física, a integração dos componentes eletrônicos e o desempenho do sistema em testes operacionais.

### 3.1 Protótipo do Veículo

O chassi do veículo, apresentado na Figura 9, foi construído através de impressão 3D em material termoplástico, garantindo leveza e resistência estrutural. A mangueira de combate a incêndio foi fixada lateralmente, conectando a bomba submersa interna ao bocal frontal.

Figura 9 – Protótipo Finalizado em Perspectiva



Fonte: Os autores (2026)

A vista superior (Figura 10) revela a segmentação interna do chassi. O compartimento frontal atua como reservatório de água, enquanto a seção traseira abriga a eletrônica de controle e as fontes de energia, protegidas por tampas removíveis que facilitam a manutenção e o reabastecimento.

Figura 10 – Vista Superior Detalhando a Segmentação do Chassi



Fonte: Os autores (2026)

### 3.2 Unidade de Controle Remoto

A unidade de controle foi projetada com foco na ergonomia e funcionalidade. Internamente (Figura 11), destaca-se a PCB de face simples confeccionada artesanalmente, que centraliza as conexões dos joysticks analógicos, do módulo NRF24L01 com antena externa e da bateria de lítio recarregável.

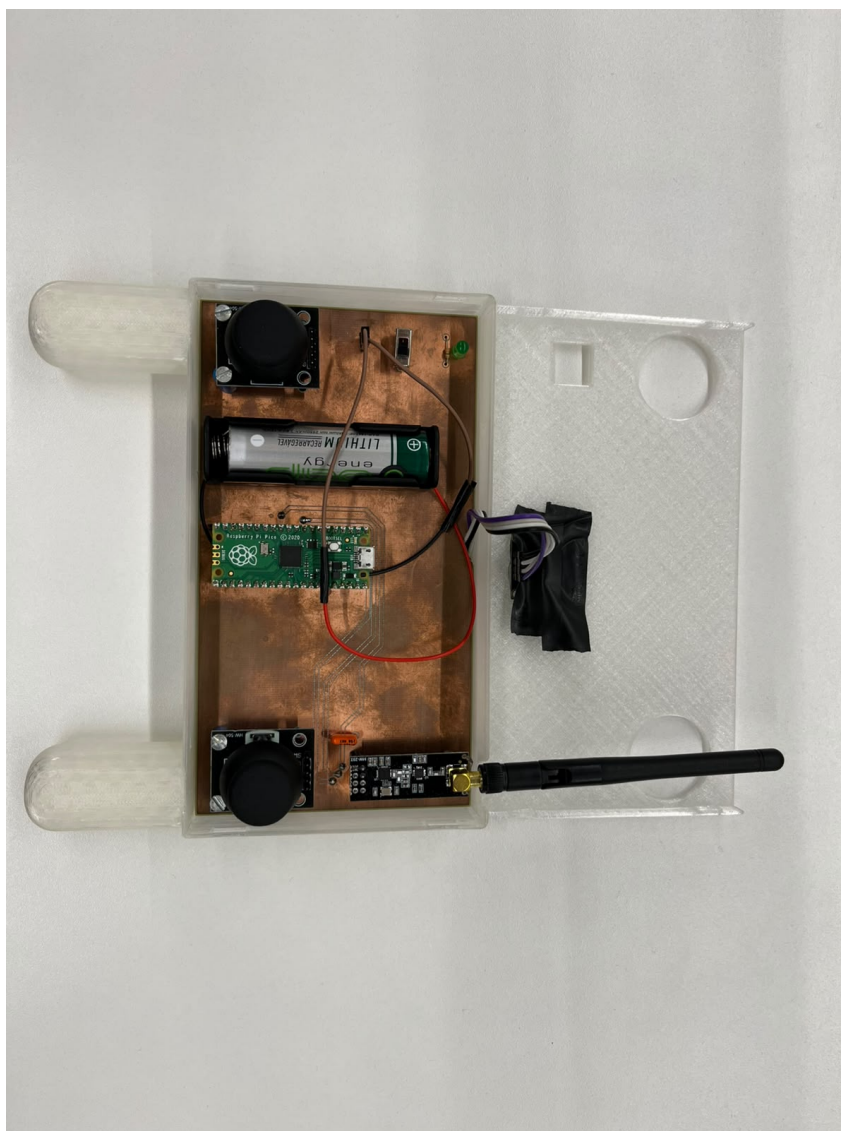
Externamente (Figura 12), o controle apresenta um layout intuitivo. O display OLED centralizado fornece informações em tempo real sobre o status da conexão e o nível do reservatório, enquanto o interruptor tipo alavanca gerencia a alimentação do sistema.

### 3.3 Integração Eletrônica e Sistema de Tração

A Figura 13 ilustra o arranjo dos componentes eletrônicos no veículo. O Raspberry Pi Pico coordena as operações, recebendo comandos via RF e controlando o driver de potência L298N para a movimentação dos motores.

O sistema de tração 4WD (Figura 14) utiliza quatro motores DC com redução, permitindo que o veículo supere pequenos obstáculos e execute manobras diferenciais. A fiação foi organizada para evitar interferências mecânicas com os eixos das rodas.

Figura 11 – Montagem Interna da Unidade de Controle

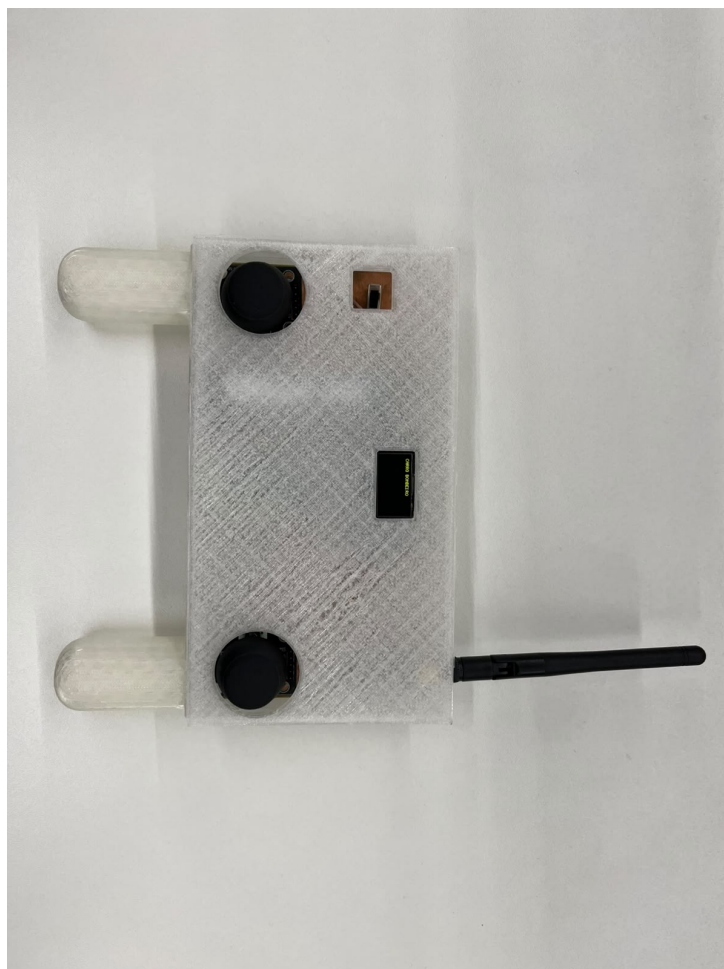


Fonte: Os autores (2026)

### 3.4 Análise de Desempenho

Os testes de campo validaram a eficiência da comunicação bidirecional. A latência percebida foi mínima, permitindo uma resposta imediata aos comandos de direção. O sistema de segurança *failsafe* atuou corretamente durante simulações de perda de sinal, interrompendo imediatamente a propulsão do veículo.

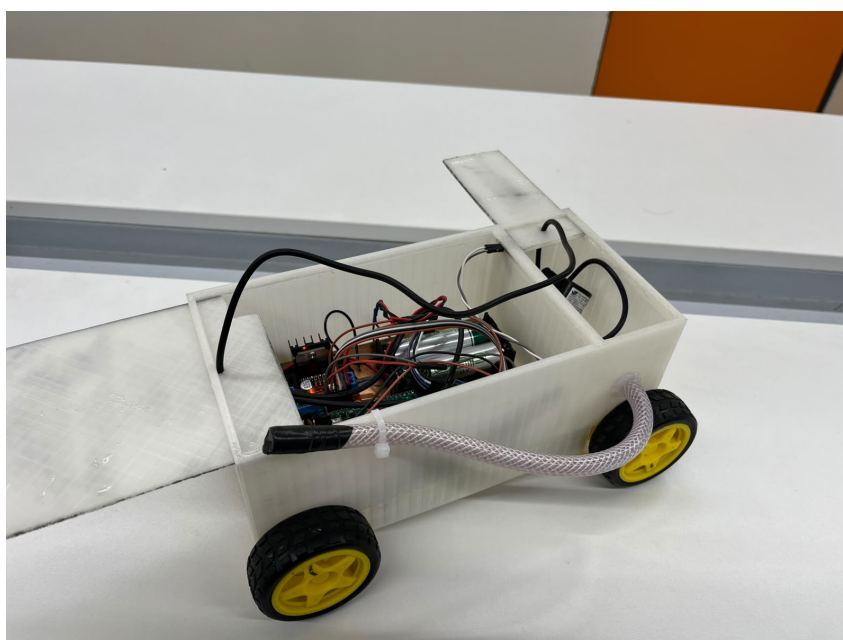
Figura 12 – Unidade de Controle Finalizada (Vista Externa)



Fonte: Os autores (2026)

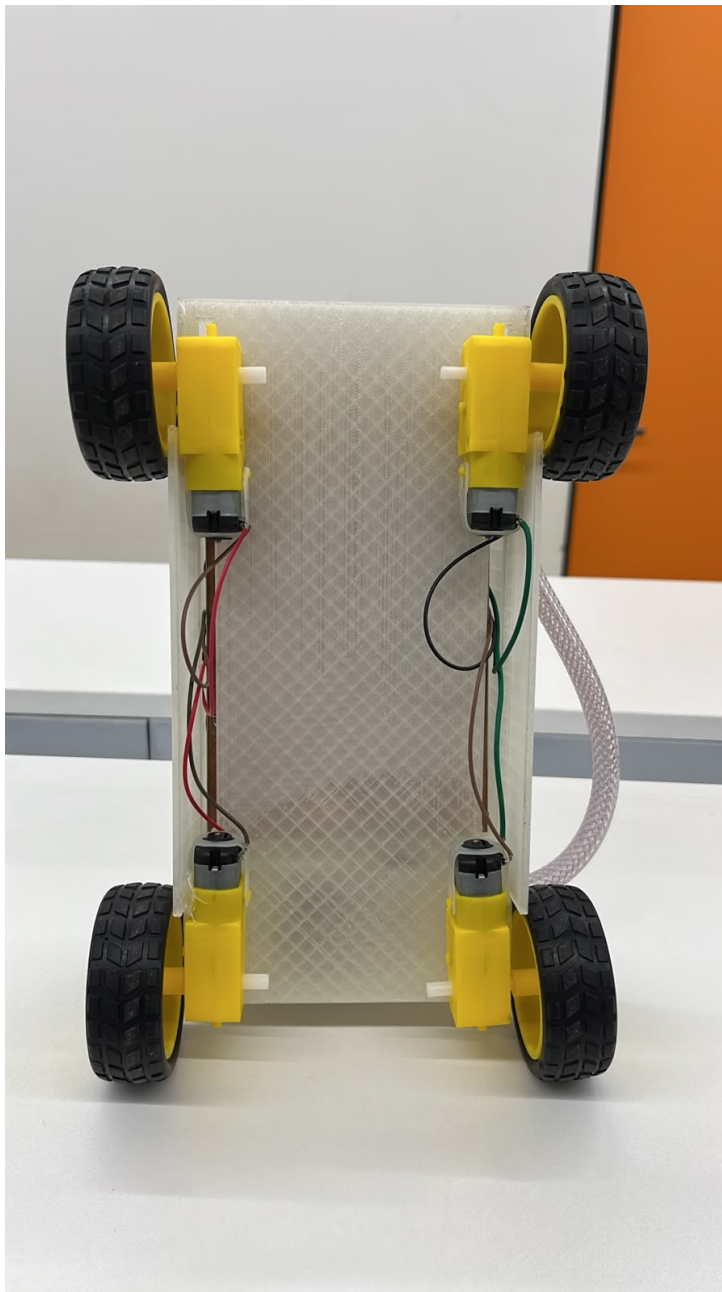


Figura 13 – Arranjo da Eletrônica de Controle no Veículo



Fonte: Os autores (2026)

Figura 14 – Sistema de Tração 4WD e Motores DC



Fonte: Os autores (2026)

## 4 Conclusão

O projeto do carro de bombeiro teleoperado foi concluído com sucesso, atingindo todos os requisitos funcionais estabelecidos inicialmente. A integração entre o microcontrolador Raspberry Pi Pico e o módulo NRF24L01 mostrou-se uma solução robusta para comunicação sem fio de baixa latência, permitindo um controle preciso mesmo em ambientes com interferência.

A implementação do algoritmo de controle *tank drive* possibilitou uma movimentação ágil e manobrável, essencial para o propósito de um veículo de intervenção. O sistema de combate a incêndio, composto pela bomba d'água e monitoramento de nível, funcionou conforme o esperado, agregando valor técnico ao protótipo através da integração de diferentes tipos de atuadores e sensores.

O desenvolvimento deste trabalho permitiu a consolidação de conhecimentos fundamentais em sistemas embarcados, incluindo programação em MicroPython, protocolos de comunicação (SPI e I2C), controle de potência com PWM e design de hardware. A experiência de projetar, montar e validar um sistema completo preparou a equipe para desafios mais complexos na área de robótica e eletrônica.

Como sugestões para trabalhos futuros, destaca-se a implementação de um sistema de controle de bocal automatizado via servomotores e a integração de sensores de chama para permitir modos de operação autônomos.

# Referências Bibliográficas

MICROPYTHON-NRF24L01 Library. 2026. Acesso em: mar. 2026. Disponível em: <https://github.com/micropython/micropython-lib/tree/master/micropython/drivers/radio/nrf24l01>>. Citado na página 8.

MICROPYTHON PROJECT. **MicroPython Documentation for RP2**. 2026. Acesso em: mar. 2026. Disponível em: <https://docs.micropython.org/en/latest/rp2/quickref.html>>. Citado na página 9.

NORDIC SEMICONDUCTOR. **nRF24L01+ Product Specification v1.0**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://www.nordicsemi.com/products/nrf24l01>>. Citado na página 8.

RASPBERRY PI LTD. **Raspberry Pi Pico Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>>. Citado na página 8.

RASPBERRY PI LTD. **RP2040 Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>>. Citado na página 8.

STMICROELECTRONICS. **L298N Dual Full-Bridge Driver Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://www.st.com/resource/en/datasheet/l298.pdf>>. Citado na página 8.

## Apêndices

# APÊNDICE A – Código Fonte do Receptor

```

1 from machine import Pin, SPI, PWM, ADC
2 import time
3
4 # =====
5 # SPI NRF24L01
6 # =====
7 spi = SPI(0,
8           baudrate=1000000,
9           polarity=0,
10          phase=0,
11          sck=Pin(6),
12          mosi=Pin(7),
13          miso=Pin(4))
14 csn = Pin(5, Pin.OUT)
15 ce = Pin(12, Pin.OUT)
16
17 # =====
18 # L298N - PONTE H
19 # =====
20 in1 = Pin(0, Pin.OUT)
21 in2 = Pin(1, Pin.OUT)
22 ena = PWM(Pin(9))
23 ena.freq(1000)
24
25 in3 = Pin(2, Pin.OUT)
26 in4 = Pin(3, Pin.OUT)
27 enb = PWM(Pin(10))
28 enb.freq(1000)
29
30 in1.value(0)
31 in2.value(0)
32 in3.value(0)
33 in4.value(0)
34 ena.duty_u16(0)
35 enb.duty_u16(0)
36
37 # =====
38 # BOMBA D'ÁGUA - GP8
39 # =====
40 bomba = Pin(8, Pin.OUT)
41 bomba.value(0)
42
43 # LED indicador

```

```
44 led = Pin(13, Pin.OUT)
45
46 # =====
47 # SENSOR DE UMIDADE
48 # =====
49 umidade_analog = ADC(Pin(27))
50 umidade_digital = Pin(22, Pin.IN)
51
52 FORA = 60000
53 SUBMERSO = 10000
54
55 def ler_umidade():
56     raw = sum(umidade_analog.read_u16() for _ in range(10)) // 10
57     nivel = (FORA - raw) * 100 // (FORA - SUBMERSO)
58     nivel = max(0, min(100, nivel))
59     digital = 1 if umidade_digital.value() == 0 else 0
60     return nivel, digital
61
62 # =====
63 # ENDEREÇOS
64 # =====
65 ADDR_CONTROLE = [0xE7, 0xE7, 0xE7, 0xE7, 0xE7]
66 ADDR_CARRO = [0xC2, 0xC2, 0xC2, 0xC2, 0xC2]
67
68 # =====
69 # DRIVER NRF24L01
70 # =====
71 class NRF24:
72     def __init__(self, spi, csn, ce):
73         self.spi = spi
74         self.csn = csn
75         self.ce = ce
76         self.csn.value(1)
77         self.ce.value(0)
78         self.init_rx()
79
80     def write_reg(self, reg, value):
81         self.csn.value(0)
82         self.spi.write(bytearray([0x20 | reg, value]))
83         self.csn.value(1)
84
85     def read_reg(self, reg):
86         self.csn.value(0)
87         self.spi.write(bytearray([reg & 0x1F]))
88         result = self.spi.read(1)
89         self.csn.value(1)
90         return result[0]
```

```
91
92     def set_addr(self, pipe_reg, addr_bytes):
93         self.csn.value(0)
94         self.spi.write(bytearray([0x20 | pipe_reg]) + bytearray(
addr_bytes))
95         self.csn.value(1)
96
97     def flush_rx(self):
98         self.csn.value(0)
99         self.spi.write(bytearray([0xE2]))
100        self.csn.value(1)
101
102     def flush_tx(self):
103         self.csn.value(0)
104         self.spi.write(bytearray([0xE1]))
105        self.csn.value(1)
106
107     def init_rx(self):
108         time.sleep_ms(100)
109         self.write_reg(0x00, 0x0B)    # Power up + RX
110         self.write_reg(0x01, 0x00)    # Sem auto-ack
111         self.write_reg(0x02, 0x01)    # Pipe 0 habilitado
112         self.write_reg(0x05, 0x02)    # Canal 2
113         self.write_reg(0x06, 0x26)    # Data rate 1Mbps
114
115         self.set_addr(0x0A, ADDR_CARRO) # RX pipe 0 addr
116         self.set_addr(0x10, ADDR_CONTROLE) # TX addr (resposta)
117         self.write_reg(0x11, 4)         # Payload pipe 0 = 4 bytes
118
119         self.flush_rx()
120         self.flush_tx()
121         self.write_reg(0x07, 0x70)
122         self.ce.value(1)
123
124     def modo_tx(self):
125         self.ce.value(0)
126         self.write_reg(0x00, 0x0A)
127         self.set_addr(0x10, ADDR_CONTROLE) # TX aponta para o controle
128         self.flush_tx()
129         time.sleep_us(150)
130
131     def modo_rx(self):
132         self.ce.value(0)
133         self.write_reg(0x00, 0x0B)
134         self.set_addr(0x0A, ADDR_CARRO)
135         self.flush_rx()
136         self.write_reg(0x07, 0x70)
```



```
137         self.ce.value(1)
138         time.sleep_us(150)
139
140     def available(self):
141         status = self.read_reg(0x07)
142         if status & 0x10:
143             self.write_reg(0x07, 0x70)
144             self.flush_rx()
145         return (status & 0x40) != 0
146
147     def receive(self):
148         self.csn.value(0)
149         self.spi.write(bytearray([0x61]))
150         data = self.spi.read(4)
151         self.csn.value(1)
152         self.write_reg(0x07, 0x40)
153         return data
154
155     def send(self, data):
156         self.mod0_tx()
157         self.csn.value(0)
158         self.spi.write(bytearray([0xA0]) + data)
159         self.csn.value(1)
160         self.ce.value(1)
161         time.sleep_us(20)
162         self.ce.value(0)
163         time.sleep_ms(5)
164         self.write_reg(0x07, 0x70)
165         self.mod0_rx()
166
167     def reset(self):
168         self.ce.value(0)
169         self.flush_rx()
170         self.flush_tx()
171         self.write_reg(0x07, 0x70)
172         self.mod0_rx()
173         print("Radio reiniciado!")
174
175     # =====
176     # FUNÇÕES DOS MOTORES
177     # =====
178     def set_motor(in_a, in_b, en_pwm, speed):
179         speed = max(-100, min(100, speed))
180         if speed >= 10:
181             in_a.value(1)
182             in_b.value(0)
183             en_pwm.duty_u16(int(speed / 100 * 65535))
```

```

184     elif speed <= -10:
185         in_a.value(0)
186         in_b.value(1)
187         en_pwm.duty_u16(int(abs(speed) / 100 * 65535))
188     else:
189         in_a.value(0)
190         in_b.value(0)
191         en_pwm.duty_u16(0)
192
193 def drive(x, y):
194     motor_esq = max(-100, min(100, y + x))
195     motor_dir = max(-100, min(100, y - x))
196     set_motor(in1, in2, ena, motor_esq)
197     set_motor(in3, in4, enb, motor_dir)
198
199 def parar():
200     set_motor(in1, in2, ena, 0)
201     set_motor(in3, in4, enb, 0)
202
203 # =====
204 # INICIALIZA RÁDIO
205 # =====
206 radio = NRF24(spi, csn, ce)
207 print("Receptor pronto, aguardando sinal...")
208 radio.modos_rx() # Garantir RX mode ao iniciar
209
210 # Debug: mostrar config inicial
211 print(">>> CONFIG INICIAL (RX):")
212 print(f"CONFIG: {radio.read_reg(0x00):#04x} | EN_RX: {radio.read_reg(0x02):#04x} | CH: {radio.read_reg(0x05):#04x}")
213
214 # =====
215 # LOOP PRINCIPAL
216 # =====
217 TIMEOUT = 500
218 TIMEOUT_RESET = 2000
219
220 ultimo_sinal = time.time()
221 primeiro_dump = True
222
223 while True:
224     if radio.available():
225         data = radio.receive()
226         t0 = time.time()
227
228         y = data[0] if data[0] < 128 else data[0] - 256
229         x = data[1] if data[1] < 128 else data[1] - 256

```

```
230     btn = data[2]
231
232     drive(x, y)
233     bomba.value(1 if btn == 0 else 0)
234     led.value(1)
235     ultimo_sinal = time.ticks_ms()
236
237     umid_pct, umid_dig = ler_umidade()
238     t1 = time.ticks_ms()
239
240     payload_umid = bytearray([
241         umid_pct & 0xFF,
242         umid_dig & 0xFF,
243         0xFF,
244         0
245     ])
246
247     radio.send(payload_umid)
248     time.sleep_ms(50) # Aumentado de 15ms → 50ms para garantir que
resposta chega ao controle
249     radio.modem_rx() # Voltar para RX mode
250     t2 = time.ticks_ms()
251
252     print(f"X: {x} | Y: {y} | BOMBA: {1 if btn == 0 else 0} | "
253           f"Nivel: {umid_pct}% | {'AGUA' if umid_dig == 1 else 'SECO'
254           },}")
255
256     print(f"RESPOSTA_TX | Umidade: {umid_pct}% | Tempo ate envio: {
time.ticks_diff(t1, t0)}ms | Total: {time.ticks_diff(t2, t0)}ms")
257
258     if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > TIMEOUT:
259         parar()
260         bomba.value(0)
261         led.value(0)
262
263     if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > TIMEOUT_RESET:
264         radio.reset()
265         ultimo_sinal = time.ticks_ms()
266
267     time.sleep_ms(10)
```

# APÊNDICE B – Código Fonte do Transmissor

```

1 from machine import Pin, ADC, SPI, I2C
2 import time
3
4 # =====
5 # CONFIG GPIO
6 # =====
7 joy1_x = ADC(27)
8 joy1_y = ADC(26)
9 button = Pin(19, Pin.IN, Pin.PULL_UP)
10 led     = Pin(25, Pin.OUT)
11 umidade_atual = 100
12
13 # =====
14 # I2C + DISPLAY OLED SSD1306
15 # =====
16 i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
17
18 class SSD1306:
19     def __init__(self, i2c, addr=0x3C, width=128, height=64):
20         self.i2c     = i2c
21         self.addr     = addr
22         self.width    = width
23         self.height   = height
24         self.pages    = height // 8
25         self.buffer   = bytearray(self.pages * width)
26         self._init_display()
27
28     def _cmd(self, cmd):
29         self.i2c.writeto(self.addr, bytearray([0x00, cmd]))
30
31     def _init_display(self):
32         for cmd in [
33             0xAE, 0x20, 0x00, 0xB0, 0xC8,
34             0x00, 0x10, 0x40, 0x81, 0xFF,
35             0xA1, 0xA6, 0xA8, 0x3F, 0xA4,
36             0xD3, 0x00, 0xD5, 0xF0, 0xD9,
37             0x22, 0xDA, 0x12, 0xDB, 0x20,
38             0x8D, 0x14, 0xAF
39         ]:
40             self._cmd(cmd)

```

```

41
42     def show(self):
43         for page in range(self.pages):
44             self._cmd(0xB0 + page)
45             self._cmd(0x00)
46             self._cmd(0x10)
47             self.i2c.writeto(self.addr,
48                             bytearray([0x40]) + self.buffer[page * self.width:(page
+ 1) * self.width])
49
50     def fill(self, color):
51         val = 0xFF if color else 0x00
52         self.buffer = bytearray([val] * len(self.buffer))
53
54     def pixel(self, x, y, color):
55         if 0 <= x < self.width and 0 <= y < self.height:
56             page = y // 8
57             bit = y % 8
58             idx = page * self.width + x
59             if color:
60                 self.buffer[idx] |= (1 << bit)
61             else:
62                 self.buffer[idx] &= ~(1 << bit)
63
64     FONT = {
65         ' ': [0,0,0,0,0],
66         'A': [0x7E,0x09,0x09,0x09,0x7E],
67         'B': [0x7F,0x49,0x49,0x49,0x36],
68         'C': [0x3E,0x41,0x41,0x41,0x22],
69         'D': [0x7F,0x41,0x41,0x22,0x1C],
70         'E': [0x7F,0x49,0x49,0x49,0x41],
71         'F': [0x7F,0x09,0x09,0x09,0x01],
72         'G': [0x3E,0x41,0x49,0x49,0x7A],
73         'H': [0x7F,0x08,0x08,0x08,0x7F],
74         'I': [0x41,0x7F,0x41,0x00,0x00],
75         'J': [0x20,0x40,0x41,0x3F,0x01],
76         'K': [0x7F,0x08,0x14,0x22,0x41],
77         'L': [0x7F,0x40,0x40,0x40,0x40],
78         'M': [0x7F,0x02,0x0C,0x02,0x7F],
79         'N': [0x7F,0x04,0x08,0x10,0x7F],
80         'O': [0x3E,0x41,0x41,0x41,0x3E],
81         'P': [0x7F,0x09,0x09,0x09,0x06],
82         'Q': [0x3E,0x41,0x51,0x21,0x5E],
83         'R': [0x7F,0x09,0x19,0x29,0x46],
84         'S': [0x46,0x49,0x49,0x49,0x31],
85         'T': [0x01,0x01,0x7F,0x01,0x01],
86         'U': [0x3F,0x40,0x40,0x40,0x3F],

```

```

87     'V': [0x1F,0x20,0x40,0x20,0x1F],
88     'W': [0x3F,0x40,0x38,0x40,0x3F],
89     'X': [0x63,0x14,0x08,0x14,0x63],
90     'Y': [0x07,0x08,0x70,0x08,0x07],
91     'Z': [0x61,0x51,0x49,0x45,0x43],
92     '0': [0x3E,0x51,0x49,0x45,0x3E],
93     '1': [0x00,0x42,0x7F,0x40,0x00],
94     '2': [0x42,0x61,0x51,0x49,0x46],
95     '3': [0x21,0x41,0x45,0x4B,0x31],
96     '4': [0x18,0x14,0x12,0x7F,0x10],
97     '5': [0x27,0x45,0x45,0x45,0x39],
98     '6': [0x3C,0x4A,0x49,0x49,0x30],
99     '7': [0x01,0x71,0x09,0x05,0x03],
100    '8': [0x36,0x49,0x49,0x49,0x36],
101    '9': [0x06,0x49,0x49,0x29,0x1E],
102    ':': [0x00,0x36,0x36,0x00,0x00],
103    '-': [0x08,0x08,0x08,0x08,0x08],
104    '%': [0x23,0x13,0x08,0x64,0x62],
105 }
106
107 def text(self, string, x, y, color=1):
108     for ch in string.upper():
109         glyph = self.FONT.get(ch, self.FONT[' '])
110         for col_idx, col in enumerate(glyph):
111             for row in range(8):
112                 self.pixel(x + col_idx, y + row, (col >> row) & 1 if
color else 0)
113                 x += 6
114                 if x > self.width:
115                     break
116
117 # Inicializa display
118 display = SSD1306(i2c)
119 display.fill(0)
120 display.text("CARRO", 30, 20)
121 display.text("BOMBEIRO", 16, 36)
122 display.show()
123 time.sleep(2)
124
125 # =====
126 # SPI NRF24L01
127 # =====
128 spi = SPI(0,
129           baudrate=1000000,
130           polarity=0,
131           phase=0,
132           sck=Pin(6),

```

```
133         mosi=Pin(7),
134         miso=Pin(4))
135 csn = Pin(5, Pin.OUT)
136 ce  = Pin(9, Pin.OUT)
137
138 # =====
139 # ENDEREÇOS
140 # =====
141 ADDR_CONTROLE = [0xE7,0xE7,0xE7,0xE7,0xE7]
142 ADDR_CARRO    = [0xC2,0xC2,0xC2,0xC2,0xC2]
143
144 # =====
145 # DRIVER NRF24L01
146 # =====
147 class NRF24:
148     def __init__(self, spi, csn, ce):
149         self.spi = spi
150         self.csn = csn
151         self.ce  = ce
152         self.csn.value(1)
153         self.ce.value(0)
154         self.init_tx()
155
156     def write_reg(self, reg, value):
157         self.csn.value(0)
158         self.spi.write(bytearray([0x20 | reg, value]))
159         self.csn.value(1)
160
161     def read_reg(self, reg):
162         self.csn.value(0)
163         self.spi.write(bytearray([reg & 0x1F]))
164         result = self.spi.read(1)
165         self.csn.value(1)
166         return result[0]
167
168     def set_addr(self, pipe_reg, addr_bytes):
169         self.csn.value(0)
170         self.spi.write(bytearray([0x20 | pipe_reg]) + bytearray(
171 addr_bytes))
172         self.csn.value(1)
173
174     def flush_rx(self):
175         self.csn.value(0)
176         self.spi.write(bytearray([0xE2]))
177         self.csn.value(1)
178
179     def flush_tx(self):
```

```
179         self.csn.value(0)
180         self.spi.write(bytearray([0xE1]))
181         self.csn.value(1)
182
183     def init_tx(self):
184         time.sleep_ms(100)
185         self.write_reg(0x00, 0x0A)
186         self.write_reg(0x01, 0x00)
187         self.write_reg(0x02, 0x01)
188         self.write_reg(0x05, 0x02)
189         self.write_reg(0x06, 0x26)
190         self.set_addr(0x10, ADDR_CARRO)
191         self.set_addr(0x0A, ADDR_CONTROLE)
192         self.write_reg(0x11, 4)
193         self.flush_rx()
194         self.flush_tx()
195         self.write_reg(0x07, 0x70)
196
197     def modo_rx(self):
198         self.ce.value(0)
199         self.write_reg(0x00, 0x0B)
200         self.write_reg(0x02, 0x01)
201         self.set_addr(0x0A, ADDR_CONTROLE)
202         self.flush_rx()
203         self.write_reg(0x07, 0x70)
204         self.ce.value(1)
205         time.sleep_us(150)
206
207     def modo_tx(self):
208         self.ce.value(0)
209         self.write_reg(0x00, 0x0A)
210         self.flush_tx()
211         time.sleep_us(150)
212
213     def send(self, data):
214         self.modo_tx()
215         self.csn.value(0)
216         self.spi.write(bytearray([0xA0]) + data)
217         self.csn.value(1)
218         self.ce.value(1)
219         time.sleep_us(15)
220         self.ce.value(0)
221         time.sleep_ms(3)
222         self.write_reg(0x07, 0x70)
223         self.modo_rx()
224
225     def available(self):
```



```

226     status = self.read_reg(0x07)
227     if status & 0x10:
228         self.write_reg(0x07, 0x70)
229         self.flush_rx()
230     return (status & 0x40) != 0
231
232     def receive(self):
233         self.csn.value(0)
234         self.spi.write(bytearray([0x61]))
235         data = self.spi.read(4)
236         self.csn.value(1)
237         self.write_reg(0x07, 0x40)
238         return data
239
240     def dump_regs(self):
241         print("=== DUMP REGISTRADORES ===")
242         print(f"CONFIG      (0x00): {self.read_reg(0x00):#04x}")
243         print(f"EN_AA      (0x01): {self.read_reg(0x01):#04x}")
244         print(f"EN_RXADDR  (0x02): {self.read_reg(0x02):#04x}")
245         print(f"RF_CH      (0x05): {self.read_reg(0x05):#04x}")
246         print(f"RF_SETUP   (0x06): {self.read_reg(0x06):#04x}")
247         print(f"STATUS     (0x07): {self.read_reg(0x07):#04x}")
248         print(f"RX_PW_P0   (0x11): {self.read_reg(0x11):#04x}")
249         self.csn.value(0)
250         self.spi.write(bytearray([0x10]))
251         tx_addr = self.spi.read(5)
252         self.csn.value(1)
253         print(f"TX_ADDR    (0x10): {[hex(b) for b in tx_addr]}")
254         self.csn.value(0)
255         self.spi.write(bytearray([0x0A]))
256         rx0_addr = self.spi.read(5)
257         self.csn.value(1)
258         print(f"RX_ADDR_P0 (0x0A): {[hex(b) for b in rx0_addr]}")
259         print("=====")
260
261     # =====
262     # INICIALIZA RÁDIO
263     # =====
264     radio = NRF24(spi, csn, ce)
265     print(">>> DUMP INICIAL (modo TX):")
266     radio.dump_regs()
267
268     # =====
269     # FUNÇÕES
270     # =====
271     def read_joystick adc):
272         return adc.read_u16()

```

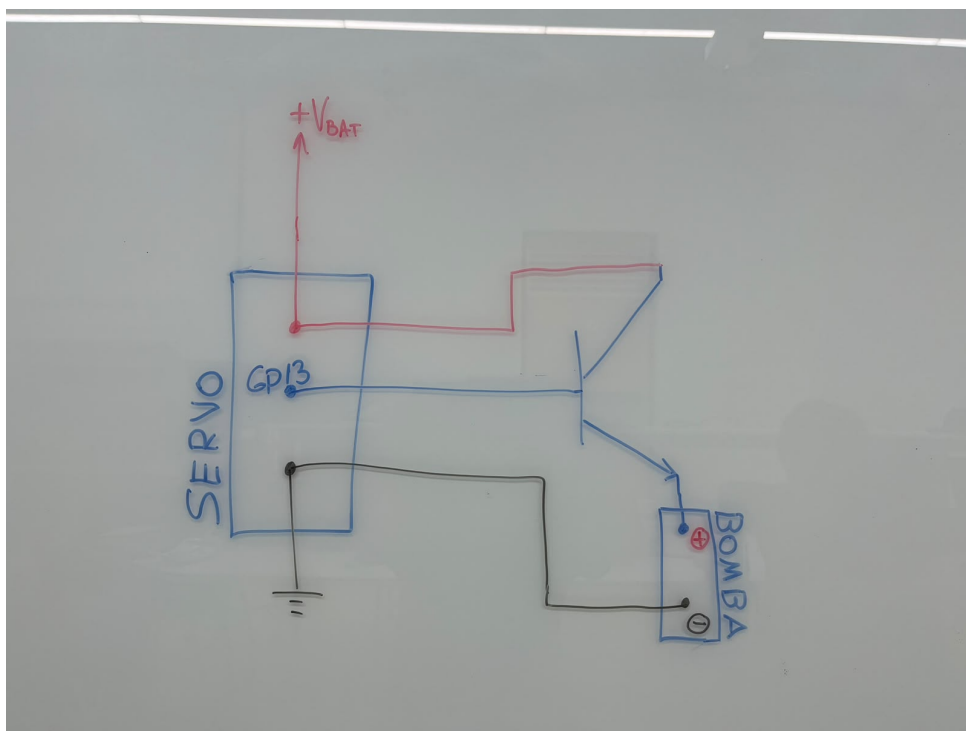
```
273
274 def normalize(value):
275     return int((value - 32768) / 32768 * 100)
276
277 # =====
278 # LOOP PRINCIPAL
279 # =====
280 INTERVALO_TX = 80
281 JANELA_RX    = 25
282
283 ultimo_tx = time.ticks_ms()
284
285 while True:
286
287     if time.ticks_diff(time.ticks_ms(), ultimo_tx) >= INTERVALO_TX:
288
289         x_norm = -normalize(read_joystick(joy1_x))
290         y_norm = -normalize(read_joystick(joy1_y))
291
292         btn = button.value()
293
294         led.value(1 if btn == 0 else 0)
295
296         # Determina status (AGUA se >= 30%, SECO abaixo)
297         umidade_status = "AGUA" if umidade_atual >= 30 else "SECO"
298
299         payload = bytearray([
300             x_norm & 0xFF,
301             y_norm & 0xFF,
302             btn,
303             0
304         ])
305
306         radio.send(payload)
307
308         ultimo_tx = time.ticks_ms()
309
310         deadline = time.ticks_add(time.ticks_ms(), JANELA_RX)
311         recebeu = False
312
313         while time.ticks_diff(deadline, time.ticks_ms()) > 0:
314             if radio.available():
315                 resp = radio.receive()
316                 if resp[2] == 0xFF:
317                     print("RX OK | Umidade real:", resp[0])
318                     recebeu = True
319                     break
```

```
320         time.sleep_ms(1)
321
322         if not recebeu:
323             print(f"Sem resposta RF | Umidade SIM: {umidade_atual}% [{umidade_status}]")
324
325         # ---- Atualiza OLED ----
326         display.fill(0)
327
328         display.text("CARRO BOMBEIRO", 4, 0)
329
330         display.text("X:" + str(x_norm), 0, 18)
331         display.text("Y:" + str(y_norm), 0, 30)
332
333         display.text(
334             "BTN:" + ("ON" if btn == 0 else "OFF"),
335             0, 42
336         )
337
338         display.text(
339             "UM:" + str(umidade_atual) + "% " + umidade_status,
340             0, 54
341         )
342
343         display.show()
344
345     time.sleep_ms(1)
```

## APÊNDICE C – Modificação Técnica da Bomba d'Água

A Figura 15 detalha a alteração realizada no circuito de acionamento da bomba d'água para permitir o controle via transistor, garantindo o acionamento adequado a partir do sinal de controle do microcontrolador.

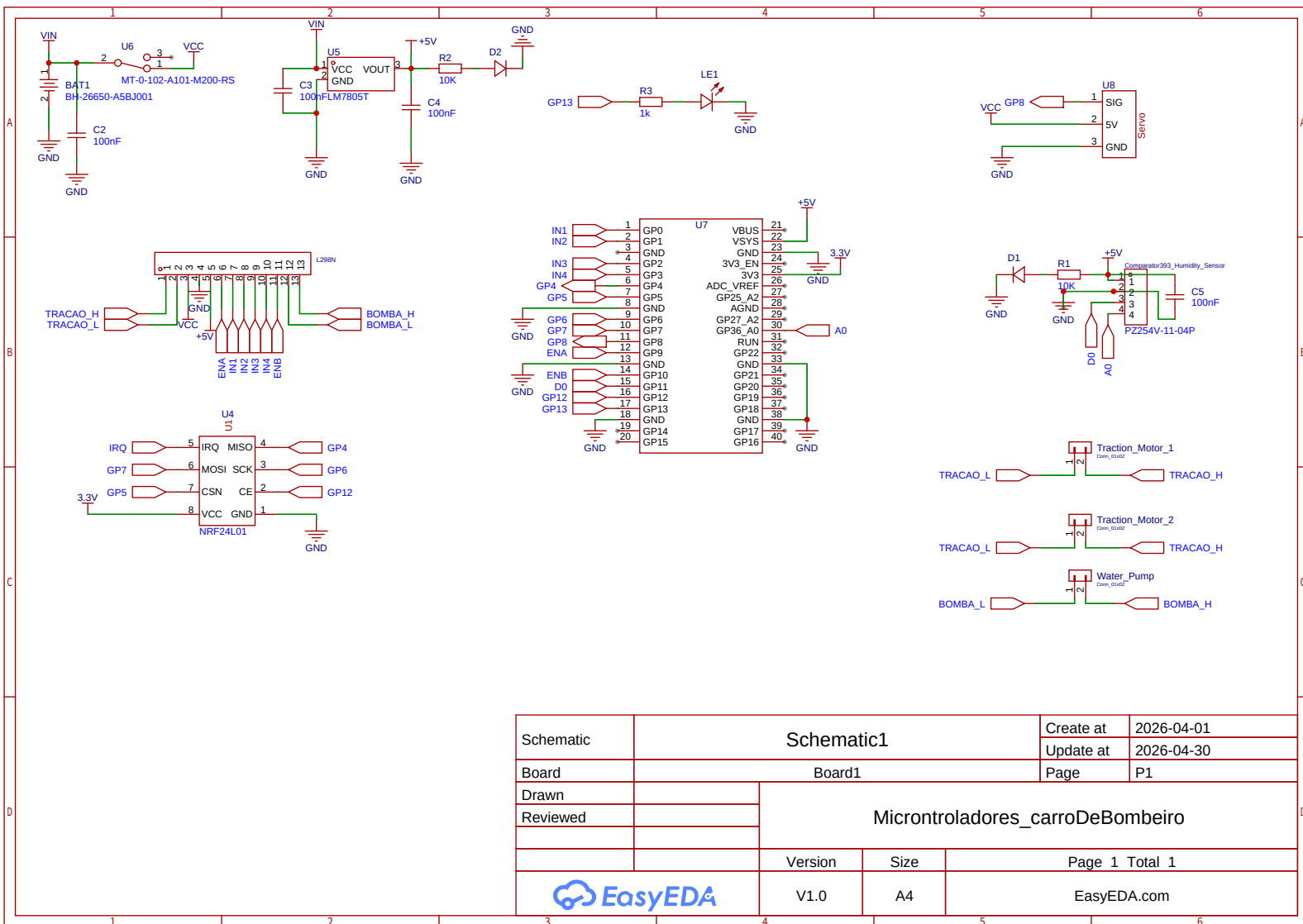
Figura 15 – Esquema da Modificação da Bomba d'Água



Fonte: Os autores (2026)

## Anexos

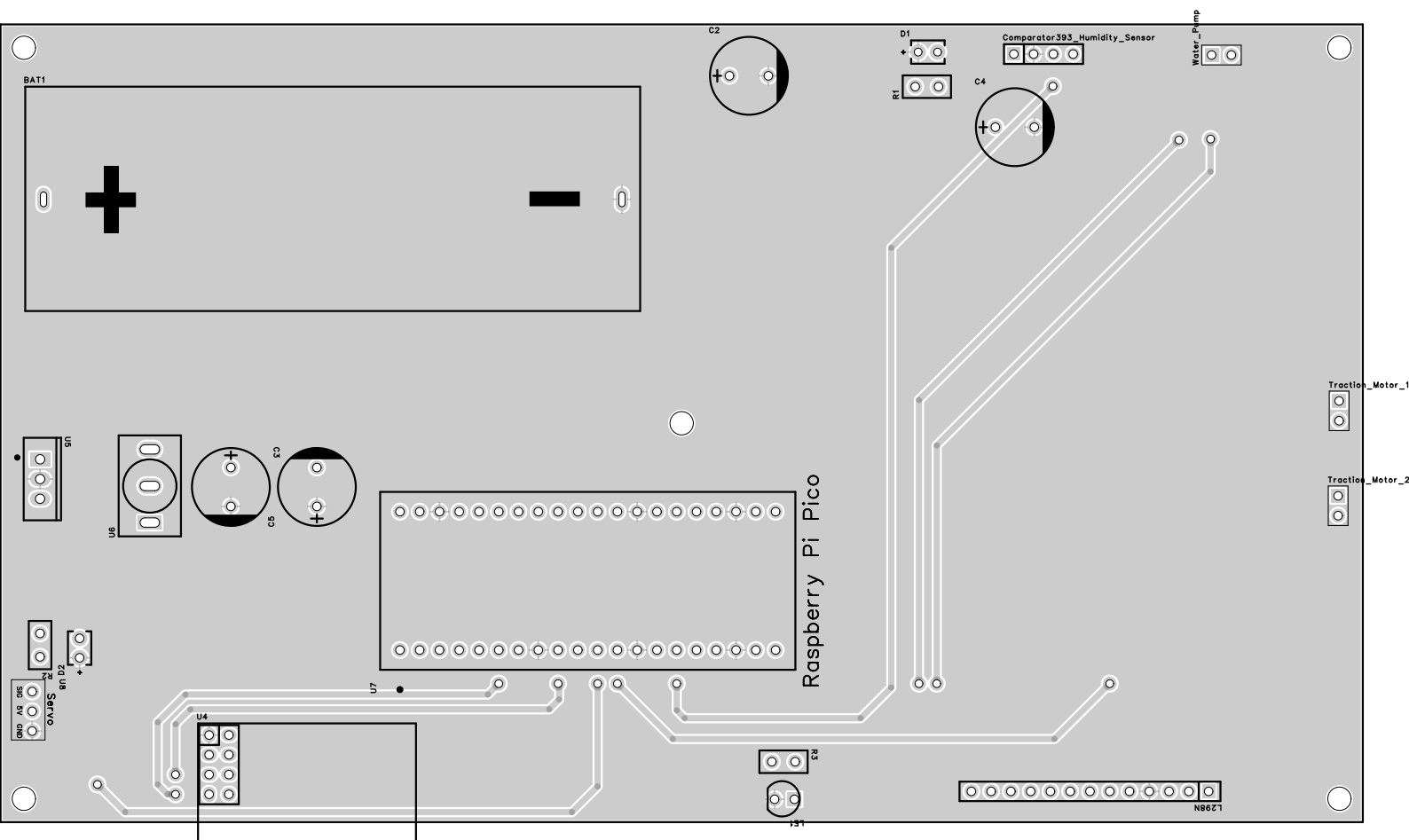
## ANEXO A – Esquema Elétrico do Carro

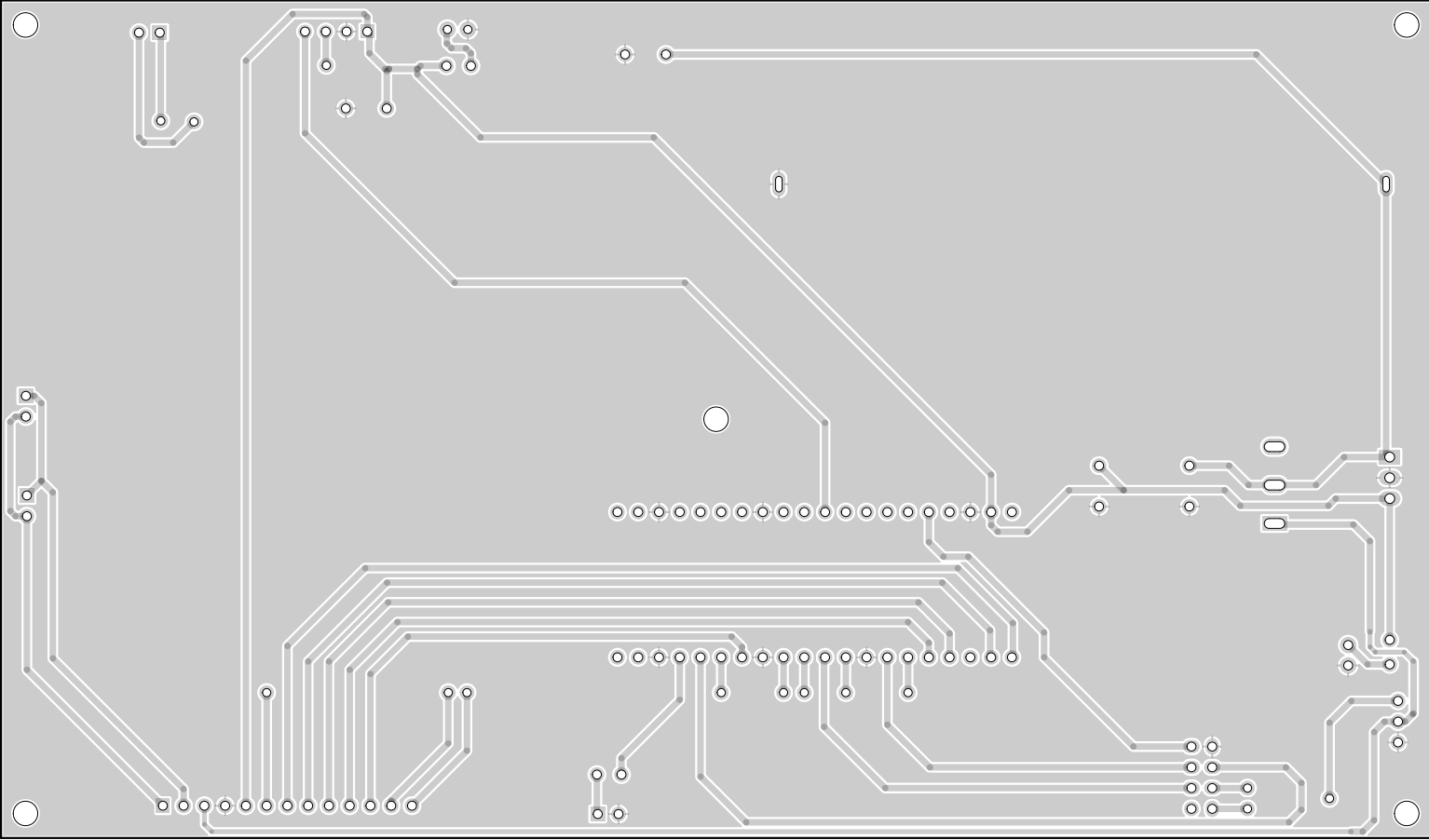


|           |                                  |             |                |
|-----------|----------------------------------|-------------|----------------|
| Schematic | Schematic1                       | Create at   | 2026-04-01     |
|           |                                  | Update at   | 2026-04-30     |
| Board     | Board1                           | Page        | P1             |
| Drawn     | Microntroladores_carroDeBombeiro |             |                |
| Reviewed  |                                  |             |                |
|           | Version                          | Size        | Page 1 Total 1 |
| EasyEDA   |                                  | V1.0        | A4             |
|           |                                  | EasyEDA.com |                |

## ANEXO B – Layout da PCB do Carro

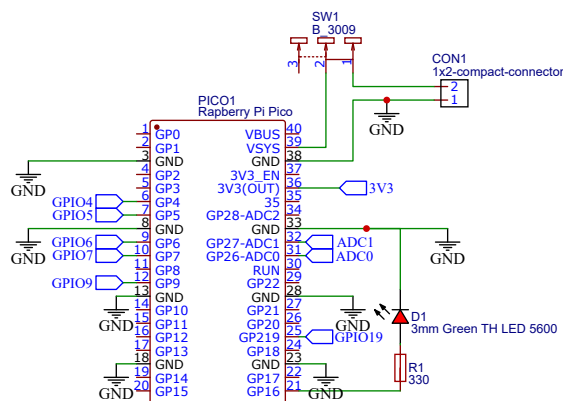
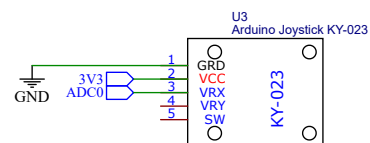
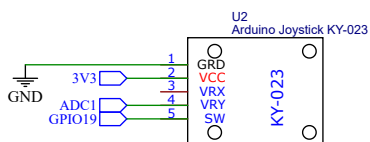
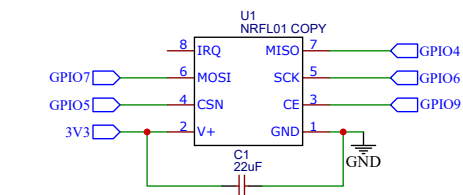








## ANEXO C – Esquema Elétrico do Controle



|                       |  |                    |
|-----------------------|--|--------------------|
| TITLE: Sheet_1        |  | REV: 1.0           |
| Company: Your Company |  | Sheet: 1/1         |
| Date: 2026-04-01      |  | Drawn By: jgforuci |

## ANEXO D – Layout da PCB do Controle

